(1.1)	(1.2)	(1.3)	(1.4)	(1.5)
C	B	B	B	A
(2.1)	(2.2)	(2.3)	(2.4)	(2.5)
ABC	ABD	ABE	ABC	BD

1 Single Choices

1. You have to sort n data with very limited extra memory ($\Theta(1)$ extra memory every case). Choose sort algorithm(s) which is(are) appropriate.

(1) merge sort, (2) insertion sort (3) bubble sort (4) quick sort

A. (1)(2)
B. (1)(2)(3)
C. (2)(3)
D. (2)(3)(4)
E. (2)(4).

2. In merge sort, if the merge function uses $if\ (a1[i1] < a2[i2])$ and otherwise takes the element from the right array. What is the stability of this implementation?

A. Stable.
B. Unstable.
C. Stable only when the two sub-arrays have same number of inversions .
D. Stable only when the two sub-arrays are of equal length.

3. Using half-open intervals $[first, last)$ with $mid = (first + last)/2$, which combination of calls is correct?

A. merge_sort(first, mid); merge_sort(mid + 1, last); merge(first, mid + 1, last)
B. merge_sort(first, mid); merge_sort(mid, last); merge(first, mid, last)
C. merge_sort(first, mid - 1); merge_sort(mid, last); merge(first, mid - 1, last)
D. merge_sort(first, last - 1); merge_sort(last - 1, last); merge(first, last - 1, last)

4. What is the expected number of pairs of inversions in an array of length n?

A. $\Theta(n)$ B. $\Theta(n^2)$ C. $\Theta(n \log n)$ D. $\Theta(\log(n!))$

5. Which of the following is correct?

A. When array A contains distinct elements and is arranged in descending order, the time complexity of quick sort(always choose the last element as pivot) is $\Theta(n^2)$.
B. All sorting algorithms must be $\omega(n)$.
C. Any comparison-based sorting algorithm can be represented by a comparison tree. Worst-case running time can be less than the height of the tree
D. There doesn't exist a sorting algorithm takes $\theta((n+1)!)$ time.

3 Sort Practice (8 pts)

(a) Run Insertion Sort for array $\{1, 6, 2, 7, 8, 3, 5, 4\}$. Write down the array after each outer iteration. (2 pts)

```
for(int k = 1; k < n; k++){
    for(int j = k; j > 0; j--){
        if( array[j- 1] > array[j] )
            swap(array[j- 1], array[j]);
        else
            break;
        }
    print(array);
}
```

```
0: {1,6,2,7,8,3,5,4}
1: {1,6,2,7,8,3,5,4}
2: {1,2,6,7,8,3,5,4}
3: {1,2,6,7,8,3,5,4}
4: {1,2,3,6,7,8,5,4}
5: {1,2,3,5,6,7,8,4}
6: {1,2,3,4,5,6,7,8}
```

(b) From list (diagram) to arrays (fill numbers). (2 pts)

Run flagged bubble Sort for the array in (a). Write down the array **after each** outer iteration.

```
for(int i = n-1; i > 0; i--){
    int max_t = array[0];
    bool sorted = true;
    for(int j = 1; j <= i; j++){
        if(array[j] < max_t){
            array[j-1] = array[j];
            sorted = false;
        }
        else{
            array[j-1] = max_t;
            max_t = array[j];
        }
    }
    array[i] = max_t;
    print(array);
    if (sorted)
        break;
}
```

start {1,6,2,7,8,3,5,4}
i=7 {1,2,6,7,3,5,4,8}
i=6 {1,2,6,3,5,4,7,8}
i=5 {1,2,3,5,4,6,7,8}
i=4 {1,2,3,4,5,6,7,8}

(c) (2 pts) Run Merge Sort for this array. Write down the array **after** each merge and underline the sub-array being merged. Note that insertion sort are only used when for sub-array of size 2 or less.

start: $\{1, 6, 2, 7, 8, 3, 5, 4\}$ (16,27 sort)
Merge 1: $\{\underline{1, 2, 6, 7}, 8, 3, 5, 4\}$ (83,54 sort)
Merge 2: $\{1, 2, 6, 7, \underline{3, 4, 5, 8}\}$
Merge 3: $\{\underline{1, 2, 3, 4, 5, 6, 7, 8}\}$

(d) (2 pts) Run Quick Sort for this array. Choose the last entry as pivot in this question. Write down the array **after** each quick sort operation(move all entries smaller than the pivot to left and move all entries larger than the pivot to right, then move the pivot. Please refer to the implementation in ppt.) and underline the sub-array. Note that insertion sort are only used when for sub-array of size 2 or less.

start: $\{1, 6, 2, 7, 8, 3, 5, 4\}$
pivot=4 $\{\underline{1, 2, 3}, 4, \underline{8, 6, 5, 7}\}$
pivot=3 $\{\underline{1, 2}, 3, 4, 8, 6, 5, 7\}$
pivot=7 $\{1, 2, 3, 4, \underline{6, 5}, 7, \underline{8}\}$
final: $\{1, 2, 3, 4, 5, 6, 7, 8\}$

4 Insertion Sort using Linked List (4 pts)

We have learnt the insertion sort implementation using array. In this question, you are required to implement insertion sort using single linked list. Since it is not easy to traverse single linked list from back to front, we can traverse from front to back instead if it is needed.

Fill in the blanks to complete the algorithm. Please note that there is at most one statement (ended with ;) in each blank line.

```
struct ListNode
{
    int val;
    ListNode *next;
    ListNode(int x) : val(x), next(nullptr) {}
};

ListNode* insertionSort(ListNode* head) {
    //if linked list is empty or only contains 1 element, return directly
    if(!head || !(head->next)){return head;}

    //create dummy node
    ListNode *dummy = new ListNode(-1);
    dummy->next = head;

    //split linked list into sorted list and unsorted list
    ListNode *tail = head; //tail of sorted list
    ListNode *sort = head->next; //head of unsorted list

    //insertion sort
    while(sort)
    {
       if(sort->val < tail->val)
       {
           ListNode *ptr = dummy;
           while(ptr->next->val < sort->val) ptr = ptr->next;
           // Your code HERE!
           // line1
           // line2
           // line3
           // line4
       }
       else
       { //no need to insert
           tail = tail->next;
           sort = sort->next;
       }
    }
```

```
    ListNode *ans = dummy->next;
    delete dummy; dummy = nullptr;
    return ans;
}
```

Fill:

(1) $tail->next = sort->next;$

(2) $sort->next = ptr->next;$

(3) $ptr->next = sort;$

(4) $sort = tail->next;$

5 Quick Sort (6 pts)

(a) (2 pt)If we use randomized quick-sort (i.e., randomly choosing pivots) to sort the array $[3, 9, 6, 2, 1, 5, 8, 0]$, the probability of 2 and 5 being compared is _____. $\frac{2}{3}$

(b) (4 pts) Fill in the blanks to complete the algorithm. Please note that there is at most one statement in each blank line.

```
// Median-of-3: ensure a[first] <= a[mid] <= a[last-1],
//then swap the median to the end as pivot
template <typename T>
T pick_pivot_median3(T* a, int first, int last) { // [first, last)
    int mid = first + (last - first) / 2;
    // TODO(1): after three swaps, ensure a[first] <= a[mid] <= a[last-1]
    if (/* TODO(1a) */) std::swap(a[first], a[mid]);
    if (/* TODO(1b) */) std::swap(a[first], a[last - 1]);
    if (/* TODO(1c) */) std::swap(a[mid], a[last - 1]);
    std::swap(a[mid], a[last - 1]);
    return a[last - 1];
}

// Find in [i, last-1) the first index with a[index] >= pivot
template <typename T>
int find_next_ge(const T& pivot, T* a, int i, int last) {
    while (/* TODO(2) */) {
        ++i;
    }
    return i;
}

// Find in (first, j] the last index with a[index] < pivot
template <typename T>
int find_prev_lt(const T& pivot, T* a, int j, int first) {
    while (j > first && !(a[j] < pivot)) {
        --j;
    }
    return j;
}

// Quicksort: sort the half-open range [first, last)
template <typename T>
void quick_sort(T* a, int first, int last) {
    while (last - first > QS_INSERTION_THRESHOLD) {
        // Choose pivot (median moved to a[last-1])
        T pivot = pick_pivot_median3(a, first, last);

        // Left keeps < pivot, right keeps >= pivot
```

```
        int low  = find_next_ge(pivot, a, first, last);
        int high = find_prev_lt(pivot, a, last - 2, first);

        while (low < high) {
            std::swap(a[low], a[high]);
            ++low;
            --high;
            low  = /* TODO(3a) */;
            high = /* TODO(3b) */;
        }

        // Place pivot into its final position;
        std::swap(/* TODO(4a) */,/* TODO(4b) */);
        int pivot_pos = low;

        // Tail recursion elimination: recurse on the smaller side first
        int left_size  = pivot_pos - first;
        int right_size = last - (pivot_pos + 1);

        if (left_size < right_size) {
            if (left_size>1) quick_sort(a, first, pivot_pos);
            first = pivot_pos+1;
        } else {
            if (right_size>1) quick_sort(a, pivot_pos + 1, last);
            last = pivot_pos;
        }
    }

    if (last - first > 1) {
        insertion_sort(a, first, last);
    }
}
```

Fill:

(1a) $a[first] > a[mid]$

(1b) $a[first] > a[last-1]$

(1c) $a[mid] > a[last-1]$

(2) $i < last \&\& a[i] < pivot$

(3a) $find_next_ge(pivot, a, low, last)$

(3b) $find_prev_lt(pivot, a, high, first)$

(4a) $a[low]$

(4b) $a[last-1]$

6 Choosing Appropriate Sorting Algorithms (12 pts)

For each of the following scenarios, choose a sorting algorithm (from either bubble sort, insertion sort, or merge sort) that best applies, and justify your choice. Each sort may be used more than once. If you find that multiple sorts could be appropriate for a scenario, identify their pros and cons, and choose the one that best suits the application. State and justify any assumptions you make. "Best" should be evaluated by asymptotic running time.

(a) Suppose you are given a data structure `D` maintaining an extrinsic order on n items, supporting two standard sequence operations: `D.get` at(i) in worst-case $\Theta(1)$ time and `D.set` at(i, x) in worst-case $\Theta(n \log n)$ time. Choose an algorithm to sort the items in `D` in place best.

(a)Merge

Choose Merge with extra space of o(1) to make D.set less.

(b) Suppose you have a static array A containing pointers to n comparable objects, pairs of which take $\Theta(\log n)$ time to compare. Choose an algorithm to sort the pointers in A so that the pointed-to objects appear in non-decreasing order with minimum time cost.

(b) Merge

The cost of the comparison is high, so we should choose the Merge sort for its fewer comparisons.

(c) Suppose you have a sorted array A containing n integers. Now suppose someone performs some $\log \log n$ swaps between pairs of adjacent items in A so that A is no longer sorted. Choose an algorithm to best re-sort the integers in A.

(c) Insertion

The array is almost sorted. Insertion is fast in this case, while Merge or Bubble would be slower.